

Know Who You Are: Earning Roles from Situated Peer Judgments

Anonymous Author(s)

Submission Id: INTERNAL PRE-RESULTS – NOT FOR SUBMISSION

Abstract

Multi-agent systems must assign responsibility before they know which collaborator will make a dependency usable, yet downstream failure entangles producer defects with recipient integration. Existing reputation, acceptance, and terminal-reward schemes collapse these sources: they can rank agents, but cannot tell whether a recipient’s concrete use of a delivery is evidence about the producer or about the recipient. We introduce *earning roles*, a responsibility-aware protocol that binds a versioned delivery to the recipient’s observable use, filters it through an ownership gate, publishes the resulting evidence before the next assignment, and applies delayed selected-only credit only to that future opportunity. The protocol makes a falsifiable prediction: under matched information, candidate sets, and complete cost, situated evidence should improve the assignment of unseen responsibilities only when it adds producer-attributable information beyond acceptance and terminal outcomes. We formalize the event and read-cut boundaries and specify candidate ArtifactRole and PeerSelect evaluation tracks with same-information baselines, root-level splits, and latency, forgetting, attribution, and cost measures; general expertise, universal reputation, and unbounded workflow discovery remain outside the claimed scope.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; *Cooperation and coordination*.

Keywords

multi-agent systems, role formation, peer judgment, online adaptation, responsibility

ACM Reference Format:

Anonymous Author(s). 2027. Know Who You Are: Earning Roles from Situated Peer Judgments. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), Hanoi, Vietnam, 3–7 May 2027*, IFAAMAS, 9 pages.

Internal pre-results manuscript. The current scientific release gate is closed. The tables below define the experiment before confirmation data are observed; numerical result cells are intentionally blank.

1 Introduction

In dependency-bearing multi-agent workflows, a collaborator’s value is observed through another agent’s attempt to use its artifact. How can collaborators earn future responsibilities from the

judgments of agents who use their work? Consider two constructed queue-processing cases with the same rejection and failed team outcome. In one, a producer violates the priority-module contract; in the other, the module is correct but the recipient implements its consumer incorrectly. The same feedback calls for different responsibility assignments. Observing only rejection and terminal failure cannot distinguish the cases; the recipient’s account of what it used and why it failed may provide the missing evidence.

This example isolates responsibility confounding: the same feedback can describe producer correctness, recipient integration, or neither. The recipient has direct evidence of the delivery’s use, but its judgment can also reflect its own integration difficulties. Learning from that judgment requires separating these sources and testing whether the resulting evidence predicts useful future assignments. A later outcome can inform subsequent decisions; it cannot retrospectively alter the information available to an already sealed assignment.

This paper studies a sharper question: *when can the agent that actually receives and uses a delivery provide reliable evidence about the producer’s future responsibility?* The question has a necessary sequence:

delivery → recipient use and judgment → producer attribution
→ future assignment → unseen quality and cost. (1)

A final score cannot by itself identify which producer caused a defect. An acceptance bit cannot distinguish a useful artifact from an easy integration. A recipient’s normal integration work cannot automatically become a producer penalty. A role posterior that never changes a later pre-execution assignment is not role formation.

The central idea is to make the recipient’s local observation a public, version-bound evidence object with a responsibility boundary. A producer is credited only when the delivery was observed, the recipient action is traceable to that delivery, and a contract check excludes recipient-owned work. Evidence is published without silently updating the policy. Before a later task starts, a legal owner reads the snapshot and the selector seals its assignment and propensity. Only after the later task has an independent outcome can a delayed credit update occur. This separates what a recipient saw from what the future responsibility actually produced, and makes the separation observable in the ledger.

Figure 1 shows the loop; its colors separate situated observation, public evidence, and the later decision.

We name this protocol *earning roles*. The name describes a measurable process: a collaborator earns responsibility opportunities through attributable evidence that changes future assignment under a fixed information and cost contract. We call this responsibility-aware role evidence (RARE); the name refers to the loop’s evidence semantics, not to a particular updater. RARE is defined by the judgment–attribution–assignment loop, not by a particular representation, neural head, RLS update, or periodic refit. The scientific claim to test is that this loop identifies useful role evidence beyond



This work is licensed under a Creative Commons Attribution International 4.0 License.

raw acceptance, terminal-only outcomes, and same-information contextual trust.

The paper makes four concrete contributions as a testable design.

- We define an event and information contract that separates producer quality, recipient action, downstream adoption, final outcome, and UNKNOWN while preserving the exact delivery, version, read cut, and propensity.
- We give a two-stage role-evidence protocol: publication is immutable and does not change the policy; a later assignment is sealed before execution; delayed selected-only credit can affect only the future opportunity it evaluates. These boundaries make noninterference and idempotent replay testable.
- We specify an ArtifactRole track for responsibility diagnosis and a secondary PeerSelect mechanism track, with matched baselines, root-level splits, and complete resource accounting to distinguish information value from closed-loop team benefit.
- We state the falsification boundary. If recipient evidence has no out-of-sample information, if a same-information trust baseline explains the effect, if attribution is unsafe, or if quality gains disappear at matched complete cost, the role-formation claim does not pass.

The present evidence boundary is deliberate. Development traces qualify parts of the protocol and expose scorer and ownership failures, but they do not establish role learning; the claims below therefore remain tied to the independently frozen experiment and its evidence level. The remainder of the paper positions the gap, defines the event and information contract, specifies the protocol and matched comparisons, and then gives the research questions, reserved result matrix, current evidence, and limits.

2 Related Work and Positioning

The nearest work differs from our question along three axes: what is observed, when state changes, and whose contribution receives credit. Table 1 makes that boundary explicit. Partner-selection and reputation methods motivate adaptive choice, while graph and role systems motivate changing organization; neither, by itself, identifies whether a concrete producer delivery caused a recipient’s action. A contextual-trust or delayed-bandit adapter supplies a strong same-information alternative, not a synonym for the proposed mechanism.

The distinction from ordinary online learning is temporal and semantic. A source judgment is available before a later assignment, but the later quality outcome is available only after that assignment has executed. A target outcome may update later decisions once it arrives. Using it to reconstruct an earlier assignment would violate that assignment’s information boundary; treating it as a label of the source artifact would also require a separate attribution argument. Our protocol makes the two events explicit and reports the service cost of maintaining the separation. Bandit estimators therefore provide useful update comparators; they are not themselves the claimed contribution.

3 Problem Formulation

3.1 Objects and information boundary

At episode t , the task supplies context x_t and a legal candidate set C_t . The selector chooses one candidate before any candidate in C_t executes:

$$a_t \sim \pi_\theta(\cdot \mid x_t, C_t, R_t), \quad a_t \in C_t, \quad (2)$$

where R_t is the public evidence snapshot available at read cut w_t . Only the selected candidate is charged for execution. The candidate emits a versioned delivery o_t with digest d_t ; a recipient j_t reads that delivery and takes action m_t . The recipient may accept, revise, reject, or be unable to complete the integration.

The smallest auditable event is

$$e_t = (x_t, C_t, a_t, o_t, j_t, J_t, m_t, Q_t, Y_t, \tau_t, c_t), \quad (3)$$

where J_t is the recipient judgment, Q_t is the producer-contract check, Y_t is an independent later or terminal outcome when the task defines one, τ_t is feedback-arrival metadata, and c_t is the complete cost record. The event also stores the candidate version, task/root identifier, artifact digest, feature-schema version, selection probability $p_t = \pi_\theta(a_t \mid x_t, C_t, R_t)$, read-cut digest, and evidence lineage.

Concrete instance. In the queue case, x_t is the JSON task request, $C_t = \{A, B\}$ is the legal producer menu, and $a_t = B$ selects producer B . The delivery o_t is B ’s versioned priority-module artifact with digest d_t ; recipient field $j_t = k$ records k ’s import, test, or repair action as m_t ; Q_t checks only the producer-owned queue and priority paths; Y_t is the later consumer outcome; and c_t contains model, tool, token, and wall-clock cost.

A repair to k ’s consumer path is recorded in m_t and sets the responsibility gate to $G_t = 0$ rather than changing B ’s producer evidence.

The policy may consume only public events whose arrival is at or before its read cut. Hidden gold, a private scorer, a future target outcome, another policy’s state, and an unexecuted candidate cannot affect selection, retry, stopping, or update. A missing or non-attributable event is UNKNOWN, not a negative label. This boundary is the difference between a role signal and a convenient retrospective score.

3.2 Responsibility gate

For a source episode, define $G_t = 1$ only when all of the following hold: (i) the producer contract or a pre-registered source defect is uniquely bound to the producer; (ii) the recipient inspected o_t and its action is traceable to o_t ; (iii) recipient-owned integration paths are empty; (iv) J_t, m_t, Q_t , identifiers, versions, and digests are complete; and (v) the source event has not already been credited. Otherwise $G_t = 0$ and the source cannot create producer evidence.

This gate distinguishes four observations that are often collapsed: a producer-content defect, a recipient’s own integration work, a sink or final-task failure, and an unresolved attribution. The public record stores the reason for $G_t = 0$. This makes safety measurable: a method that refuses ambiguous evidence may have a higher UNKNOWN rate while producing fewer false producer penalties.

Figure 2 shows the episode, public-evidence, and local-decision lanes.

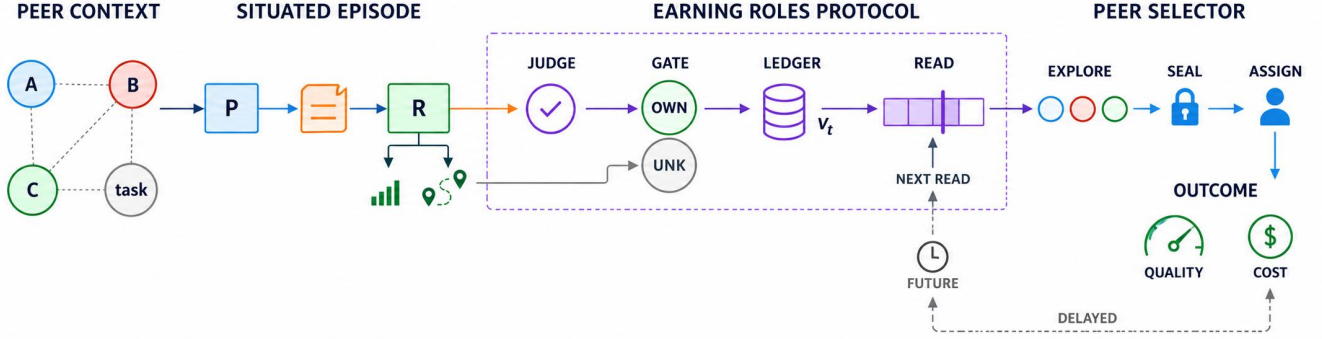


Figure 1: The earning-roles protocol. Orange marks situated observation; green marks producer-attributable evidence, while the **UNKNOWN** branch remains audit-only; blue marks the sealed assignment and scored outcome. Gray dashed feedback is delayed and selected-only, entering a future state and next read cut without changing historical evidence or a sealed assignment. Peer-context links are descriptive, not causal.

Representative method and problem	Observed signal and update	Responsibility / credit semantics	Experimental object	Difference / test here
Curiosity-Driven Partner Selection; Defection at First Sight [4, 5]	Interaction outcomes; partner state changes after social-game feedback	Peer-level interaction payoff; the cited formulations do not expose a producer-recipient artifact gate	Convention or dilemma games	Versioned delivery use and producer attribution
Reputation as a Solution to Cooperation Collapse [3]	Shared assessments and cooperation history; pooled reputation update	Broad reputation/cooperation score; the cited formulation does not expose producer and recipient ownership fields	Language-agent social cooperation	Public responsibility-safe evidence and explicit UNKNOWN
AgentNet; GPTSwarm [6, 8]	Agent traces, graph structure, or routing outcomes; organization changes over time	Route/role utility; the cited formulations do not define a producer-artifact credit label	Graph or workflow coordination	Recipient action bound to one delivery and future-owner assignment
Dynamic Role Assignment; MoRSE [2, 7]	Task context and role/subtask performance; roles or experts are reassigned	Role/expert utility; the cited formulations do not expose producer-artifact attribution inside recipient integration	Debate or mixture-of-experts workflows	Responsibility gate, cross-owner propagation, and delayed correction
Finite-time multi-armed bandit [1]	Selected-arm rewards; update after an observed arm payoff	Arm-level reward; no producer-recipient ownership contract	Pre-execution selection under observed payoff	Classical selected-reward control; contextual and delayed adapters require separate qualification

Table 1: Novelty boundary used in the paper. The rows are comparison families, not claims that each source implements the same task. A faithful adapter must preserve the listed visible information, arrival order, selected-only denominator, and complete cost before entering the scientific matrix.

4 Earning Roles: A Responsibility-Aware Protocol

4.1 Two-stage evidence and update

The protocol has two independent interfaces:

$$\text{publish}(e_t) \rightarrow \{\text{RoleEvidence}, \text{UNKNOWN}, \text{invalid}\}, \quad (4)$$

$$\text{choose}(x_t, C_t, R_t) \rightarrow (a_t, p_t, \delta_t), \quad (5)$$

$$\text{validate}(j_t) \rightarrow \{\text{credit}, \text{UNKNOWN}, \text{invalid}\}, \quad (6)$$

$$\text{update}(\text{credit}) \rightarrow \{\text{updated once}, \text{no-op}\}. \quad (7)$$

Publication may change the public input to a later assignment, but it does not call the persistent updater. For a later target assignment j , credit is legal only if the assignment was written before target selection and task start, the selected candidate and propensity match the sealed preview, the target delivery and outcome are complete, and the assignment has not already been credited. The update then

evaluates that future responsibility opportunity, never the source episode.

The event-time contract is therefore

$$\begin{aligned} \text{source selection} &\rightarrow \text{delivery} \rightarrow (Q, J, m, Y) \rightarrow G \\ &\rightarrow \text{evidence publication} \rightarrow \text{assignment} \rightarrow \text{target outcome} \quad (8) \\ &\rightarrow \text{delayed selected-only update.} \end{aligned}$$

A correction creates a superseding evidence version and affects only future snapshots. Duplicate, late, out-of-order, resource-failed, or invalid events are idempotent no-ops or explicit **UNKNOWN** records. Ledger replay must reproduce the same state digest and choice sequence.

4.2 Evidence representation and candidate score

A minimal public state for producer v in context r is

$$R(v, r) = (n(v, r), s(v, r), u(v, r)), \quad (9)$$

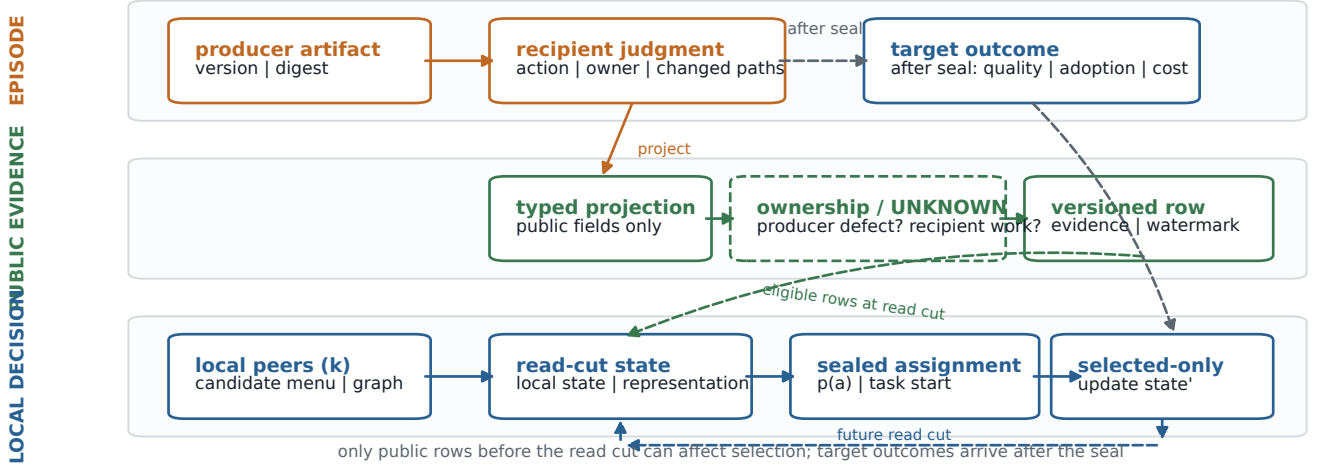


Figure 2: Evidence-to-role state. The episode lane records what the producer and recipient actually do; the public-evidence lane projects only typed, responsibility-checked fields; and the local-decision lane reads eligible rows at a read cut before sealing an assignment. Target outcomes arrive after the seal and update only the matching selected opportunity. The representation and updater remain replaceable interfaces.

where n counts eligible evidence, s accumulates responsibility-safe positive/negative labels, and u records unresolved or rejected evidence for audit. A candidate score can combine a task representation $h(x, v, r)$ with the public evidence term:

$$\text{score}(v \mid x, r) = w^\top h(x, v, r) + \beta \frac{s(v, r)}{n(v, r) + \alpha}. \quad (10)$$

For the queue instance, r is the priority-contract context, $h(x, B, r)$ is the fixed task-candidate representation for producer B , and n, s, u are counts over the versioned events that passed the gate. A recipient-only repair increases u or creates an explicit UNKNOWN record; it cannot increase s for B .

The representation h , updater, and constants are replaceable implementation conditions rather than the scientific object. The score is a contract-level construction, not a theorem or a claim that one backbone is optimal. The selector uses a pre-registered exploration floor, records p_t , and uses preview-assignment-commit so that writing a future assignment cannot resample the selected candidate.

The same protocol yields three observable predictions. First, responsibility safety is measured by false producer-attribution and UNKNOWN rates under recipient-only interventions. Second, temporal validity is measured by read-cut and assignment-order invariants and by the absence of target-outcome fields at selection. Third, bounded online service is measured by per-event update latency, backlog, state bytes, and complete cost. These predictions map to the RQ2–RQ4 endpoints; they are not assumptions that the score automatically satisfies.

4.3 Executable selector and update contract

The protocol becomes runnable with a small stateful selector whose interfaces are fixed before confirmation data are read. We use a

frozen text encoder only to place a task, a candidate’s public description, and its eligible evidence in a common space; the encoder is not updated by a feedback event. To prevent a profile from silently reusing the label that trains the decision head, each candidate has a timestamped profile snapshot and a post-snapshot evidence delta:

$$P_{v,k} = \text{summarize}\{e_i : G_i = 1, t_i < \kappa_k\}, \quad (11)$$

$$D_{v,k,t} = \text{summarize}\{e_i : G_i = 1, \kappa_k \leq t_i \leq w_t\}, \quad (12)$$

$$z_t(v) = [\phi(x_t) \parallel \phi(P_{v,k}) \parallel \phi(D_{v,k,t}) \parallel b(v)]. \quad (13)$$

Here κ_k is the last profile cut, w_t is the legal read cut, ϕ is frozen, and $b(v)$ contains version and uncertainty features that are already public at w_t . An empty profile or delta is represented explicitly; it is not replaced by a future label. The split is a provenance rule, not a tuning convention: an event can enter exactly one of P , D , or a later target outcome.

The candidate distribution is

$$q_t(v) = \text{softmax}_{v \in C_t} (g_\psi(z_t(v)) + \theta_v^\top \phi(x_t)), \quad a_t \sim q_t, \quad (14)$$

where g_ψ is a fixed small head and θ_v is a per-candidate residual that is allowed to change only after an independent outcome for a previously selected assignment arrives. The head, temperature, exploration floor, clipping rule, and learning rate are configuration fields, not values selected after seeing confirmation results. The experiment compares the full state against profile-only, delta-only, residual-only, and no-history versions, so a gain cannot be attributed to the word “fusion” alone.

At runtime the selector executes five auditable operations. (1) It appends a source judgment to an immutable ledger and applies the ownership gate; rejected or incomplete records become UNKNOWN. (2) It advances a read watermark and materializes a state snapshot, recording the feature-schema and state digest. (3) It scores only the legal candidate menu and writes the candidate set, probabilities,

Table 2: State fields and their legal update event. A field is read only after its listed cut and never retroactively edits a sealed assignment.

Field	Source	Update boundary
Profile $P_{v,k}$	Gate-passed evidence before κ_k	Next profile cut; immutable thereafter
Delta $D_{v,k,t}$	New gate-passed evidence after κ_k	evi- Read watermark w_t
Residual θ_v	Selected target outcome Y_t	Matching delayed update only
Uncertainty $u(v)$	Unknown, duplicate, or correction records	Monotone audit update
Assignment a_t, p_t	Candidate menu and snapshot	Sealed before target execution

sampled action, and assignment digest before the target starts. (4) When the target outcome arrives, it verifies the matching assignment, selected-only eligibility, version, and idempotency key. (5) It applies one bounded residual update and emits a new state digest; profile and delta materialization wait for the next declared read cut. This sequence permits a crashed service to replay events without changing a sealed choice.

This implementation contract gives the method a measurable online meaning without claiming that one encoder or head is universally best. It also makes the difficult cases executable: a late judgment changes a future snapshot, a recipient-owned repair cannot train the producer residual, and a missing target outcome produces no update. The confirmation experiment fills in latency, state size, and predictive utility from the same ledger used to audit attribution. If a simpler calibrated comparator matches the full selector under the same read cuts and complete cost, the paper reports the protocol boundary and does not attribute the gain to the fusion architecture.

4.4 What counts as innovation

The claimed object is the closed loop in which situated judgment becomes attributable evidence, evidence changes a legal future assignment, and that assignment has an unseen quality–cost consequence. Existing representations, frozen encoders, RLS, online logistic regression, periodic refitting, caches, and prompt templates are implementation or comparison choices. The required tests are matched composition, leave-one-mechanism-out ablations, evidence-content mutation, and a same-information contextual controller. If these controls reproduce the complete result, the closed-loop mechanism has no identifiable advantage.

5 Benchmark and Baselines

5.1 Two complementary tracks

The primary track, **ArtifactRole**, is a TeamBench-derived candidate benchmark. It contains producer artifacts, recipient dependencies, use/rework/rejection actions, responsibility checks, later assignments, downstream adoption, and complete cost. Candidate roots are DIST1 queue-race (development diagnostic) and PIPE3 stream-processing (conditional confirmation); renamed seeds are not independent roots. Promotion requires material visibility, write permissions, scorer coverage, root independence, and clean replay.

The secondary **PeerSelect** track uses a pinned local graph interaction benchmark for selected-only payoff, local candidate sets,

online updates, drift, and service cost. It isolates update behavior on a smaller local-selection track, but it cannot establish situated artifact judgment or producer attribution. The two tracks share event and selector interfaces while keeping task, label, scorer, denominator, and claim separate.

5.2 Predeclared baselines

All primary arms receive the same candidate menu, initial opportunities, model/API/tool budget, exploration opportunity, feedback arrival schedule, failure handling, and complete cost accounting. Each arm reads only its legal history. Table 3 names the alternative explanations that the matrix must rule out.

RLS, online logistic/SGD, periodic refitting, and any candidate incremental updater are training comparators. They are not counted as separate selection policies and are not presented as the innovation merely because they are fast.

6 Experimental Questions and Matrix

6.1 Research questions and estimands

RQ1 (information value). Does an eligible situated judgment predict an independent producer-contract or later-use result beyond raw acceptance and terminal-only feedback? The primary estimands are out-of-sample log loss, Brier score, AUC when defined, calibration, and incremental predictive value on an independent root.

RQ2 (assignment causality). Does public evidence change an assignment made before target execution, including when the future owner did not produce the source judgment? We measure assignment-change probability, propensity-weighted future quality, adoption, rework, and complete cost. Evidence from producer B cannot be consumed as if it belonged to producer C .

RQ3 (online behavior). Does the update path satisfy the service contract under selected-only delayed feedback? We report publish, read/assignment, and update p50/p95 latency, service lag, backlog, state bytes, CPU/GPU/RAM, token/tool/API calls, and replay time.

RQ4 (stability and boundaries). How does the method behave under task or peer drift, delayed or out-of-order feedback, version replacement, recipient-only work, mixed ownership, scorer failure, and resource failure? We report old-root peak and mean forgetting, drift recovery window, UNKNOWN rate, correction consistency, and false producer attribution.

The primary unit of uncertainty is an independently reset stream. Episodes inside one stream are dependent. We use paired stream-level estimates, bootstrap or randomization intervals over streams, and pre-registered multiplicity control for the primary contrasts. Missing, invalid, or unknown events remain in the denominator and are never silently recoded as failures or successes.

Primary endpoint design. The design reserves four card identifiers: H1 uses independent-target Brier score (lower is better) for incremental situated signal; H2 uses future assignment quality–complete-cost utility (higher is better) against the same-information control; H3 uses selected-feedback update p95 latency and backlog under the fixed arrival budget (lower is better); and H4 uses false producer-attribution rate under ownership mutations (lower is better). Each card is required to fix its control, minimum practically

Arm	Read/update contract	Alternative explanation tested
Uniform	Candidate menu; fixed local-uniform sampling	Random allocation and exploration lower bound
No update	Initial prior and current context; feedback ignored	Static execution is sufficient
Raw acceptance	Recipient accept/reject only	Responsibility filtering adds no value
Terminal only	Independent final outcome only	Situated judgment adds no information
Contextual trust/bandit	Same context, judgment, propensity, delay, and budget as RARE	The effect is ordinary contextual trust or bandit learning
Pooled controller	Permitted public history under the same total capacity	A shared controller explains the gain
Closest published adapter	Faithful public-information adapter, if it passes qualification	Existing role/organization method is sufficient
RARE	Responsibility-safe evidence plus delayed selected-only credit	Candidate closed-loop mechanism

Table 3: Primary comparison boundary. A named arm enters the scientific matrix only after an executable adapter, information declaration, cost contract, and independent qualification pass.

important difference or precision target, stream-level 95% interval, and stopping rule before outcomes are observed; the current design-only manifest records which fields remain unfrozen.

Only these H1–H4 endpoints are primary for the headline interpretation. The remaining measures in the matrix are secondary or exploratory and cannot be promoted after outcomes are observed.

6.2 Complete experiment matrix

Table 4 is the execution skeleton. The result columns are deliberately empty; the confirmation manifest must bind every row to a root, split, policy, seed, budget, scorer, and raw log before any result is filled.

Figure 3 links the two tracks to the four research-question endpoints; it does not merge their labels or claims.

Table 5 reserves the endpoints that will carry the paper’s headline comparison once the confirmation manifest is complete.

6.3 Budget, statistics, and reporting

Every model request reserves attempted calls, prompt tokens, maximum completion tokens, tool time, and storage before dispatch; errors release only unused reservations and retain their observed cost. We charge producer execution, recipient integration, judgment service, communication, retries, replay, verification, and returned artifacts. Development/search cost is reported separately from deployment cost. A pooled controller receives the same total capacity, and arms that do not use a judgment still pay the declared comparison cost or use a shared judgment call whose output is masked.

For each stream we record configuration, model route and version metadata, code and data hashes, seed, hardware, timestamps, raw API responses, token usage, intermediate actions, failure traces, UNKNOWN reasons, and per-sample outcomes. We report means and medians with 95% stream-level intervals, effect sizes for pre-registered contrasts, per-root heterogeneity, ITT and per-protocol denominators, and quality–cost Pareto points. The main endpoint is assignment-level future quality–complete-cost utility; source producer score is auxiliary and cannot replace it.

For every primary contrast, the experiment card fixes the direction, minimum practically important difference or precision target, stream-level 95% interval, and stopping rule before confirmation. A row without those fields is a design specification rather than an interpretable result.

6.4 Ablations and falsification

The minimum ablation grid is: (A) no evidence/no update; (B) public evidence only, with persistent state unchanged; (C) delayed update only, without new public evidence; and (D) public evidence plus delayed update. Additional interventions remove responsibility filtering, selected-only restriction, delay, locality, correction/replay, or the assignment binding one at a time while holding information, model, candidate menu, and budget fixed. A content mutation that changes a visible judgment while holding terminal quality fixed must change the comparator score if the comparator actually consumes that judgment.

The main claim is withdrawn if an evidence publication implicitly changes persistent policy; if a recipient repair is credited to a producer; if assignment or read-cut order is invalid; if unknown or duplicate events update state; if contextual trust explains the complete effect; or if gains disappear at matched complete cost. A null, a trade-off, or a qualified protocol result is scientifically valid, but it cannot be written as successful role formation.

Phase rule. Before selection, the policy sees only the candidate menu, public-row watermark, read-cut digest, and sealed propensity; hidden gold, unexecuted-candidate outcomes, and future target outcomes remain unavailable. During responsibility assessment, the record contains the judgment, action, owner, changed paths, ownership gate, and explicit UNKNOWN reason. After the assignment seal, only the matching target quality/cost event can produce one selected-only update; a late or duplicate event without correction lineage is not a new label.

6.5 Execution and result-filling contract

The remaining empirical work is organized as a sequence of frozen experiment cards rather than as an open-ended search. Each card names one matrix row, one structural root or PeerSelect stream, one policy configuration, and one seed set. Before dispatch, the card records the candidate menu, read-cut rule, feedback delay, model and encoder versions, prompt or feature schema, exploration floor, update rule, token and wall-clock budget, stopping rule, and primary estimand. These fields are immutable after the first confirmation call. A failed call remains an attempted call with its observed cost; it does not create a new seed or silently move a row to a cheaper condition.

RQ/cell	Track/root	Policy/intervention	Primary comparison	Measures	Result
RQ1-I	ArtifactRole / dev root	raw acceptance vs terminal only vs RARE	Same producer deliveries and recipient observations; legal feedback only	Brier/log loss, calibration, judge reliability	—
RQ1-II	ArtifactRole / confirmation root	contextual trust vs RARE	Same public read cut, delay, propensity, model and budget	Incremental prediction and cross-root transfer	—
RQ2-I	ArtifactRole / confirmation root	no evidence vs public evidence only	Preview-assignment before target execution	Assignment change, future quality, adoption, rework	—
RQ2-II	ArtifactRole / confirmation root	delayed update only vs public evidence + delayed update	Same target tasks and selected-only outcome	Future utility and complete cost	—
RQ2-III	ArtifactRole / confirmation root	RARE vs pooled controller	Same total public capacity and candidate menus	Quality-cost utility, concentration, propensity	—
RQ3-I	PeerSelect / dev streams	uniform, no-history, history-only, candidate updater	Local graph, selected-only payoff, fixed call/token budget	Regret/payoff, update p50/p95, state bytes	—
RQ3-II	PeerSelect / drift streams	updater with/without correction and replay	Pre-registered peer/task drift and delayed feedback	Recovery window, forgetting, backlog, replay equality	—
RQ4-I	ArtifactRole / all qualified roots	responsibility mutation matrix	Producer-only, recipient-only, mixed, unknown, resource failure	False attribution, UNKNOWN precision, no-op rate	—
RQ4-II	ArtifactRole / all qualified roots	leave-one-mechanism-out	Remove gate, public evidence, delay, locality, or updater	Main utility, safety, cost, interaction effect	—

Table 4: Pre-registered experiment matrix. Each row binds a research question to a root, split, policy, seed, budget, scorer, cost record, and raw event manifest before interpretation.

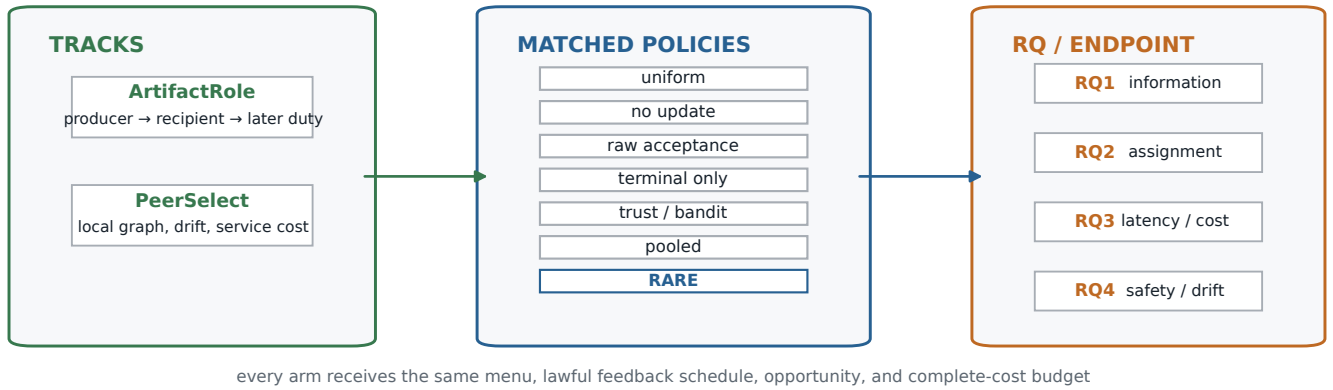


Figure 3: Experiment map. ArtifactRole measures the situated producer-recipient phenomenon; PeerSelect isolates local selection, drift, and service cost. All policy arms receive the same candidate menus, lawful feedback schedule, task opportunities, and complete-cost budget before the four research questions are scored.

Table 5: Headline endpoints. Values are supplied only by the frozen confirmation manifest; intervals are stream-level 95% intervals.

Endpoint	RARE	Best same-information arm	Difference / interval
Future quality	—	—	—
Complete cost	—	—	—
Quality-cost utility	—	—	—
Assignment change	—	—	—
Judgment Brier score	—	—	—
Update p95 / state bytes	—	—	—
Old-root forgetting	—	—	—
UNKNOWN / false attribution	—	—	—

The result-filling process has four checkpoints. First, a raw ledger checker verifies hashes, event order, selected-only eligibility, and the absence of future fields at selection. Second, a scorer produces per-episode labels, UNKNOWN reasons, costs, and stream identifiers without applying policy-specific postprocessing. Third, an analysis script computes the predeclared estimands and stream-level uncertainty from the same manifest; it writes a machine-readable result object and a human-readable audit. Fourth, the paper integrator maps only verified fields to the corresponding table cell and interpretation sentence. A cell remains TBD when a card is incomplete, a root fails qualification, or the endpoint is undefined. The integrator never replaces a missing value with zero and never edits a raw result to match a predicted direction.

This contract also fixes how negative evidence is written. A null contrast is reported as a null contrast; a safety improvement without utility gain is reported as a safety result; and a useful predictor that fails to change a future assignment is not called role formation. The final paper therefore needs no retrospective choice between

Table 6: Result-entry rules used by the confirmation cards. The table is a writing contract, not an observed result.

Checkpoint	Required artifact	Permitted paper entry
Qualification	Root, material, scorer, visibility, and replay checks	Qualified, exploratory, or UNKNOWN
Execution	Immutable event ledger and raw API/tool trace	Attempted count, failures, and cost
Analysis	Per-stream estimand, interval, and denominator	Number only if the card is complete
Integration	Hash-linked result object and reviewer audit	Table cell plus evidence-matched prose

a success story and a debugging report: the matrix, denominators, and falsification rules determine which claim survives.

7 Results and Evidence Boundary

RQ1–RQ2 (information and assignment). The confirmation card will fill the held-out Brier score, matched-information comparison, assignment-change rate, and future quality–cost utility in Tables 5 and 4: TBD. Prediction without a sealed future assignment supports an information claim only; assignment without an independent target outcome does not establish responsibility transfer.

RQ3–RQ4 (service and safety). The latency, backlog, state-size, forgetting, UNKNOWN, and false-attribution cells remain TBD. We will interpret a safe null, a quality–cost trade-off, and a qualified positive result as different claims rather than as interchangeable evidence.

Current real traces qualify engineering boundaries only. Two limited real-API chains logged source selection, recipient judgment, consumer execution, source snapshots, and a later assignment. Both consumers changed their own integration file, while a separate producer audit found priority-artifact failures; the original consumer score therefore did not measure producer correctness. The feedback mapping equaled its prior, so probabilities did not change. These traces motivate the gate and audit, but do not establish specialization, prediction, future quality, or real-time training. The agents shared a fresh prompt without persistent individual experience, and no result cell is filled from these receipts. A result without the qualified root, independent stream, complete cost, and fair adapter remains exploratory.

8 Discussion and Limitations

The mechanism is intentionally narrower than a claim that agents acquire general skills or that a workflow autonomously discovers experts. It studies responsibility opportunities in tasks with a real producer–recipient dependency. PeerSelect can isolate local online selection and drift, but it cannot replace ArtifactRole evidence. Conversely, an ArtifactRole result cannot be collapsed into a generic bandit payoff.

The main threats are judge error, selective exposure, root dependence, incomplete responsibility coverage, and cost inflation from recipient observation. The design measures judge reliability and selection propensity, preserves UNKNOWN records, and uses independent roots and streams. It cannot infer a universal role semantics from a single workflow family. If the public evidence is too sparse, the correct outcome may be a well-calibrated null or a narrower task-conditioned result.

The final backbone and updater remain open. They will be chosen only after the qualified tracks and strong same-information baselines expose a bottleneck. A frozen representation with a small incremental head, RLS or online logistic regression, periodic refitting, and a bounded candidate updater are comparison conditions. A large GPU run is justified only when a real update or representation bottleneck is measured and the experiment card fixes the challenger, budget, and confirmation split.

The deployment boundary is equally explicit. A selector may refuse to learn from an ambiguous handoff, but it must still expose that refusal to the operator and preserve the event for later audit. A selector may use a local candidate graph, but locality must be part of the card rather than an unreported source of advantage. A selector may compress old evidence, but compression must retain the source identifiers, validity interval, and state digest needed to reconstruct a decision. These requirements make the method compatible with a tool-selection service and a peer-selection service while keeping their labels separate: a tool result can be checked by an execution oracle, whereas a peer judgment must pass the producer–recipient ownership gate. The shared implementation is therefore an event and replay layer, not a shared ground truth. This distinction matters for the final claim. If the implementation provides low-latency updates but the qualified benchmark cannot identify producer responsibility, we report an executable infrastructure result and withdraw the role-learning claim. If attribution is safe but the selector does not improve future assignments, we report a calibrated evidence protocol rather than self-evolution. Both outcomes are pre-specified and remain useful to later systems.

The artifact release follows the same separation. Code, task roots, prompts, scorer versions, and environment locks are released with a manifest that points from each aggregate to its raw event file. The manifest records excluded attempts and their reasons, so a reader can distinguish a missing root from a zero-valued outcome. A replay command reconstructs read cuts and assignment seals without contacting the model provider; a separate live command is required to reproduce API latency and token cost. We will also release the exact table-generation script and the card-level decision log. These details are part of the scientific object because an online role claim is inseparable from the order in which evidence becomes visible. They let a reviewer test both positive and negative interpretations of the same stream.

9 Conclusion

This paper defines a precise object for self-evolving multi-agent roles: a recipient’s situated judgment becomes responsibility-safe evidence, that evidence changes a future assignment before execution, and the later assignment is evaluated on unseen quality and complete cost. The protocol makes each arrow auditable and gives close alternatives the same information and budget. The current draft records the method contract, benchmark and baseline matrix, statistical plan, and falsification boundary while leaving empirical cells empty. The decisive result will be whether the complete loop improves future responsibility allocation beyond equally informed alternatives without sacrificing attribution safety, online service, stability, or cost.

References

- [1] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47 (2002), 235–256. doi:10.1023/A:1013689704352
- [2] Peiwen Li, Shiyang Zhang, Yangtian Zhang, Sizhuang He, David van Dijk, and Rex Ying. 2026. MoRSE: Task-Oriented Multi-Agent System with Mixture of Role-Subtask Experts. arXiv:2608.09251 <https://arxiv.org/abs/2608.09251>
- [3] Siyue Ren, Wanli Fu, Xinkun Zou, Chen Shen, Yi Cai, Chen Chu, Zhen Wang, and Shuyue Hu. 2026. Reputation as a Solution to Cooperation Collapse in LLM-based MASs. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS. <https://www.ifaamas.org/Proceedings/aamas2026/pdfs/UEHN4980.pdf>
- [4] Benedict Russell, Chin wing Leung, and Paolo Turrini. 2026. Defection at First Sight: Learning Partner Selection in Optional Social Dilemmas without Prior Information. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS. <https://www.ifaamas.org/Proceedings/aamas2026/pdfs/IBSZ1473.pdf>
- [5] Chin wing Leung, Paolo Turrini, and Ann Nowé. 2025. Curiosity-Driven Partner Selection Accelerates Convention Emergence in Language Games. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS, 1282–1290. <https://www.ifaamas.org/Proceedings/aamas2025/pdfs/p1282.pdf>
- [6] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. 2025. AgentNet: Decentralized Evolutionary Coordination for LLM-based Multi-Agent Systems. In *Advances in Neural Information Processing Systems*, Vol. 38. https://papers.nips.cc/paper_files/paper/2025/hash/9a379c1b05793d1c42dc832269834515-Abstract-Conference.html
- [7] Miao Zhang, Junsik Kim, Siyuan Xiang, Jian Gao, and Cheng Cao. 2026. Dynamic Role Assignment for Multi-Agent Debate. arXiv:2601.17152 <https://arxiv.org/abs/2601.17152>
- [8] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. GPTSwarm: Language Agents as Optimizable Graphs. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 62743–62767. <https://proceedings.mlr.press/v235/zhuge24a.html>